CS315 · ALGORITHMS ANALYSIS & DESIGN

Practice 2 Hash Tables الحل

تدريب على 9 أسئلة · Lecture 3

محارب

01 حل Q1: Chaining

Key	22	1	13	11	24	33	18	42	31
k mod 11	0	1	2	0	2	0	7	9	9

Index	Chain
0	22 → 11 → 33
1	1
2	13 → 24
3	∅
4	∅
5	∅
6	∅
7	18
8	∅
9	42 → 31
10	∅

`22, 11, 33` كلهم باقي قسمتهم على 11 صفر، فيروحون الخانة 0 بترتيب دخولهم.

02 حل Q2: Linear probing

Key	k mod 11	Probes	Cell
22	0	0	**0**
1	1	1	**1**
13	2	2	**2**
11	0	0 → 1 → 2 → 3	**3**
24	2	2 → 3 → 4	**4**
33	0	0 → 1 → 2 → 3 → 4 → 5	**5**
18	7	7	**7**
42	9	9	**9**
31	9	9 → 10	**10**

0	1	2	3	4	5	6	7	8	9	10
22	1	13	11	24	33		18		42	31

المفتاح 33 احتاج أكبر عدد: 6 probes (من الخانة 0 إلى 5)، لأن العناصر تكتلت في أول الجدول. هذا عيب الـ linear probing.

03 Q3: Double hashing حل

Key	k mod 13	7 − (k mod 7)	Probes	Cell
14	1	7	1	**1**
27	1	1	1 → 2	**2**
40	1	2	1 → 3	**3**
19	6	2	6	**6**
53	1	3	1 → 4	**4**
7	7	7	7	**7**
66	1	4	1 → 5	**5**

0	1	2	3	4	5	6	7	8	9	10	11	12
	14	27	40	53	66	19	7					

```
27:  h = 1 taken,  d = 7 − (27 mod 7) = 7 − 6 = 1   →  (1 + 1) mod 13 = 2
```

```
40:  h = 1 taken,  d = 7 − (40 mod 7) = 7 − 5 = 2   →  (1 + 2) mod 13 = 3
```

```
53:  h = 1 taken,  d = 7 − (53 mod 7) = 7 − 4 = 3   →  (1 + 3) mod 13 = 4
```

```
66:  h = 1 taken,  d = 7 − (66 mod 7) = 7 − 3 = 4   →  (1 + 4) mod 13 = 5
```

خمسة مفاتيح لها نفس `h(k) = 1`، لكن كل واحد له خطوة `d(k)` مختلفة، فكل واحد لقى مكانه من ثاني probe.

04 حل Q4: MAD

Key	10	22	31	4	15	28	17	88	59
(2k+5) mod 11	3	5	1	2	2	6	6	5	2

```
h(10) = (20 + 5) mod 11 = 25 mod 11 = 3
```

```
h(88) = (176 + 5) mod 11 = 181 mod 11 = 5
```

:a) Chaining

Index	Chain
0	ø
1	31
2	4 → 15 → 59
3	10
4	ø
5	22 → 88
6	28 → 17
7	ø
8	ø
9	ø
10	ø

:b) Linear probing

Key	(2k+5) mod 11	Probes	Cell
10	3	3	**3**
22	5	5	**5**
31	1	1	**1**
4	2	2	**2**
15	2	2 → 3 → 4	**4**
28	6	6	**6**
17	6	6 → 7	**7**
88	5	5 → 6 → 7 → 8	**8**
59	2	2 → 3 → 4 → 5 → 6 → 7 → 8 → 9	**9**

0	1	2	3	4	5	6	7	8	9	10
	31	4	10	15	22	28	17	88	59	

05 حل Q5: مثال المحاضرة

Key	12	44	13	55	5
(3i+2) mod 7	3	1	6	6	3

Index	Chain
0	∅
1	44
2	∅
3	12 → 5
4	∅
5	∅
6	13 → 55

06 حل Q6: الحذف و AVAILABLE

a) نبحث عن 44: `44 mod 13 = 5`، الخانة 5 فيها 18، فنروح للخانة 6 ونلقاه. نحط مكانه `AVAILABLE` (مكتوبة AV في الجدول).

0	1	2	3	4	5	6	7	8	9	10	11	12
		41			18	AV	59	32	22	31	73	

```
b)   findElement(31):   31 mod 13 = 5
     probes 5 → 6 → 7 → 8 → 9 → 10
     found at cell 10, returns its element
c)   findElement(70):   70 mod 13 = 5
     probes 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12
     cell 12 is empty, returns NO_SUCH_KEY
d)   insertItem(57):    57 mod 13 = 5
     probes 5 → 6
     cell 6 stores AVAILABLE, so 57 is stored in cell 6
```

0	1	2	3	4	5	6	7	8	9	10	11	12
		41			18	57	59	32	22	31	73	

النقطة المهمة

البحث ما يوقف عند `AVAILABLE`، يوقف بس عند الخانة الفاضية. الإضافة توقف عند الاثنين.

07 حل Q7: Load factor

a) $\alpha = \frac{N}{M} = \frac{8}{13} \approx 0.615$

$$\text{Expected probes} = \frac{1}{1-\alpha} = \frac{1}{\frac{5}{13}} = \frac{13}{5} = 2.6$$

b) $\alpha = 0.9$: $\frac{1}{1-0.9} = 10$ probes

الخلاصة: كل ما قرب الـ load factor من 100% زاد عدد الـ probes بشكل كبير. عشان كذا الـ hashing سريع بشرط الجدول ما يمتلي.

c) الـ **Rehashing**: نسوي جدول جديد حجمه ضعف القديم وننقل له العناصر، فينزل الـ load factor.

08 حل Q8: Compression map

$h_2(2) = (28 + 3) \bmod 7 = 31 \bmod 7 = 3$

$h_2(5) = (70 + 3) \bmod 7 = 73 \bmod 7 = 3$

$h_2(9) = (126 + 3) \bmod 7 = 129 \bmod 7 = 3$

b) كل المفاتيح تروح للخانة `b = 3`، لأن `a mod N = 14 mod 7 = 0`. هذا يخالف شرط الـ MAD: `a mod N ≠ 0`.

09 حل Q9: أسئلة قصيرة

#	الجواب
a	1) **Chaining**: كل خانة تأشر على linked list فيها العناصر اللي جتها. بسيط لكنه يحتاج ذاكرة خارج الجدول. 2) **Linear probing** (open addressing): نحط العنصر في أول خانة فاضية بعدها، بشكل دائري. عيبه التكتل. 3) **Double hashing** (open addressing): نقفز بخطوة `d(k)` من دالة ثانية، `(i + j·d(k)) mod N`.
b	لما توزّع المفاتيح على الخانات بشكل عشوائي ومتساوي (disperse)، فتقل الـ collisions، وتكون سريعة الحساب.
c	insertItem `O(1)`، و findElement و removeElement `O(n)`.
d	الـ worst case `O(n)`، ويصير لما كل المفاتيح تتصادم. المتوقع (expected) `O(1)`.
e	**Polynomial accumulation**. الـ component sum مو جيد لأن الكلمات اللي لها نفس الحروف بترتيب مختلف تعطي نفس المجموع.
f	عشان نقدر نمر على كل خانات الجدول. و `d(k)` ما يصير تساوي صفر.